

Serverless Factory — Architecture Explained

For Learners, Developers & Demo Audiences

Table of Contents

1. [What Is This Project?](#)
 2. [The Big Picture — Simple Analogy](#)
 3. [Services Used & What Each One Does](#)
 4. [Order Pipeline — Deep Dive](#)
 5. [Resume Pipeline — Deep Dive](#)
 6. [The SNS Fan-Out Pattern Explained](#)
 7. [Why This Architecture? Real-World Impact](#)
 8. [What Happens When Things Go Wrong?](#)
 9. [Layman's Summary](#)
-

1. What Is This Project?

Serverless Factory is a web application that lets you deploy, manage, and test two fully automated cloud pipelines on AWS — without managing any servers.

Pipeline	What It Does
Order Pipeline	Validates an order → processes payment → sends email notification
Resume Pipeline	Detects a resume uploaded to S3 → processes it → stores in DB → sends notification

Both pipelines are **serverless** — meaning AWS runs the code only when needed, and you pay only for what you use.

2. The Big Picture — Simple Analogy

Think of it like a Restaurant

Restaurant Role	AWS Service	What It Does in Our System
Manager (decides what happens next)	Step Functions	Controls the order of operations

Restaurant Role	AWS Service	What It Does in Our System
Chef (does the actual cooking)	Lambda	Runs the actual business logic
Announcement speaker (tells everyone)	SNS	Broadcasts events to multiple systems
Order ticket queue (holds pending tasks)	SQS	Stores messages for processing
Trash bin for failed orders	SQS DLQ	Catches and stores failed messages
Email notification system	SES	Sends reliable transactional emails
Entrance door (triggers on arrival)	EventBridge	Triggers workflows based on events
Security guard (controls who can do what)	IAM	Manages permissions
Filing cabinet	S3	Stores files (resumes)
Database	DynamoDB	Stores structured records

3. Services Used & What Each One Does

AWS Step Functions

Simple explanation: A workflow manager that says "do step 1, then step 2, then step 3 — and if step 2 fails, do this instead."

- Keeps track of where you are in a multi-step process
- Handles retries automatically (tries again if something fails)
- Has error handling built in (catches failures, redirects to failure path)
- Visual — you can see the workflow as a diagram in AWS Console

Real-world use: Netflix uses Step Functions to manage video encoding pipelines. Airbnb uses it for booking workflows.

AWS Lambda

Simple explanation: A function that runs in the cloud. You give it code, AWS runs it when called, and you pay only for the milliseconds it runs.

- No server to manage, patch, or maintain
- Scales automatically — 1 request or 1 million requests, same code
- Each Lambda in this project does ONE job (validate, process payment, send notification)

Real-world use: Every major tech company — Coca-Cola, Lyft, iRobot — runs millions of Lambda functions daily.

AWS SNS (Simple Notification Service)

Simple explanation: A megaphone. You shout one message into it, and everyone who is listening hears it simultaneously.

- Publishers send a message to an SNS Topic
- All subscribers receive that message at the same time
- Subscribers can be: SQS queues, Lambda functions, email endpoints, HTTP webhooks
- This is called the **Fan-Out Pattern** (one in → many out)

Real-world use: Zomato publishes "order placed" to SNS → restaurant app, delivery system, analytics, and email all receive it simultaneously.

AWS SQS (Simple Queue Service)

Simple explanation: A waiting line. Messages sit in the queue until a worker picks them up and processes them.

Two types used in this project:

Queue Type	Purpose
OrdersQueue	Receives all order events from SNS (success + failure). Any system that needs order data reads from here.
Dead Letter Queue (DLQ)	Receives messages that failed processing. Acts as a safety net — nothing is ever lost.

Real-world use: Amazon's own order processing, Uber's trip event system, every bank's transaction processing system.

AWS SES (Simple Email Service)

Simple explanation: A professional email sending service. Unlike Gmail or regular email, SES is designed to send millions of transactional emails reliably.

- Sends emails directly without any subscription/confirmation step
- Emails come from your verified address
- Much more reliable than SNS email subscriptions for transactional use
- Used for: order confirmations, failure alerts, OTPs, password resets

Real-world use: Swiggy order confirmations, Flipkart shipping updates, bank OTPs — all sent via

SES or similar transactional email services.

AWS EventBridge

Simple explanation: A smart routing system that watches for events and triggers workflows.

- In this project: listens for order events and triggers the Step Functions workflow
- Can also schedule pipelines to run at specific times (like a cron job)
- Decouples the trigger from the pipeline — the pipeline doesn't need to know who triggered it

Real-world use: Shopify uses EventBridge to route order webhooks into processing pipelines.

AWS IAM (Identity and Access Management)

Simple explanation: The security system. It defines who (or what) can do what.

- Lambda function can only call the services it needs — nothing more
- Step Functions can only invoke specific Lambda functions
- EventBridge can only start specific state machines
- Principle of Least Privilege — each component gets minimum required permissions

Why it matters: If a Lambda is hacked, it can only access what IAM allows. A hacked Lambda for order processing cannot delete your S3 buckets.

AWS S3 (Simple Storage Service) — Used in Resume Pipeline

Simple explanation: A file storage system in the cloud. Like Google Drive, but for applications.

- Stores uploaded resume files
 - Triggers a Lambda automatically when a file is uploaded
 - Virtually unlimited storage, 99.999999999% durability
-

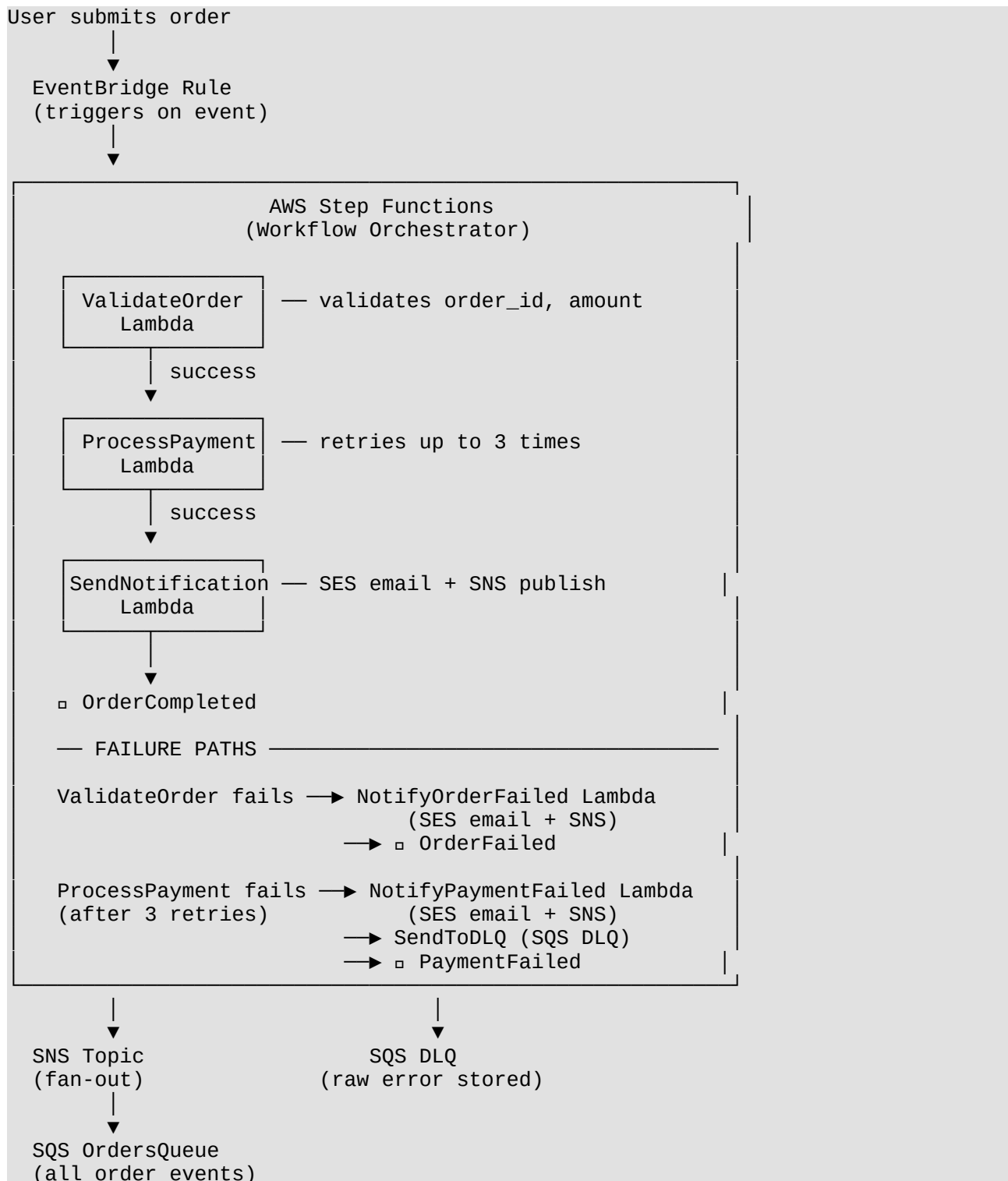
AWS DynamoDB — Used in Resume Pipeline

Simple explanation: A super-fast NoSQL database. Like a giant spreadsheet that can handle millions of reads/writes per second.

- Stores processed resume records
 - No fixed schema — flexible structure
 - Automatically scales with load
-

4. Order Pipeline — Deep Dive

Architecture Diagram



Step-by-Step Flow

Happy Path (success):

1. **User submits order** with order_id, customer_name, amount
2. **EventBridge** detects the event and triggers the Step Functions state machine

3. **ValidateOrder Lambda** checks: is `order_id` present? Is `amount > 0`?
4. **ProcessPayment Lambda** processes payment, generates a `transaction_id`
5. **SendNotification Lambda** does two things:
 - Calls **SES** → sends "Order Confirmed" email directly to the customer
 - Calls **SNS** → publishes `ORDER_COMPLETED` event with full order details
6. **SNS** delivers the event to **SQS OrdersQueue** (any downstream system reading this queue gets the event)
7. Workflow ends with **OrderCompleted**

Failure Path (payment fails):

1. **ProcessPayment Lambda** throws an exception
2. **Step Functions** automatically retries 3 times (with exponential backoff)
3. After 3 failures, **NotifyPaymentFailed Lambda** runs:
 - Calls **SES** → sends "Order Failed" alert email to customer
 - Calls **SNS** → publishes `ORDER_FAILED` event to OrdersQueue
4. **SendToDLQ** → sends raw error details to **SQS DLQ** for investigation
5. Workflow ends with **PaymentFailed**

What You Can Show in the AWS Console

Console Location	What You See
Step Functions → Executions	Visual workflow with each step highlighted green/red
SES → Sending Statistics	Email delivery activity
SNS → Topics → Subscriptions	OrdersQueue receiving events
SQS → OrdersQueue → Messages	JSON order events from SNS fan-out
SQS → OrderDLQ → Messages	Raw error details from failed payments
CloudWatch → Log Groups	Lambda execution logs for each function

5. Resume Pipeline — Deep Dive

Architecture Diagram



↳ SNS publish → "New resume processed: filename.pdf"

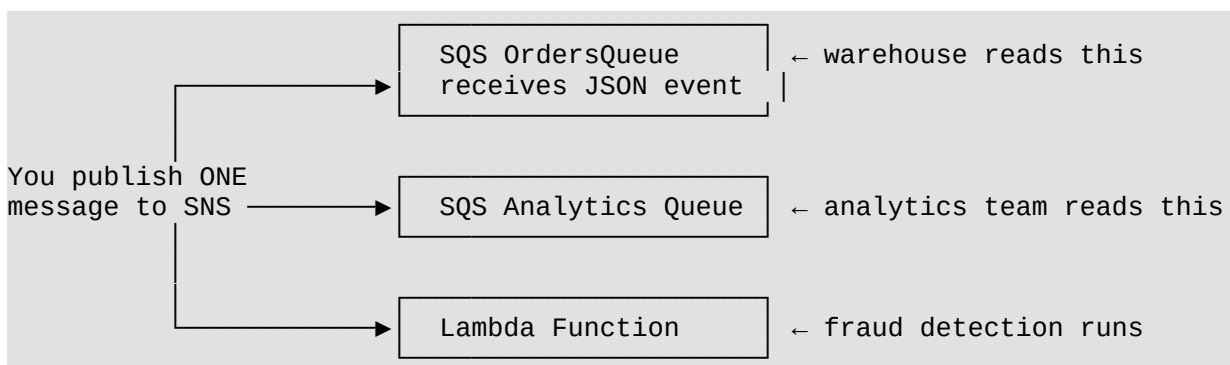
Step-by-Step Flow

1. User uploads a .pdf resume to the **S3 bucket**
2. S3 automatically triggers the **ResumeProcessor Lambda**
3. Lambda extracts metadata: filename, bucket, file size, timestamp
4. Lambda stores a record in **DynamoDB** (status: PROCESSED)
5. Lambda publishes a notification to **SNS**
6. SNS delivers to email subscriber (or SQS for further processing)
7. Lambda updates DynamoDB record (notification_status: SENT)

6. The SNS Fan-Out Pattern Explained

What Is Fan-Out?

One message published → delivered to multiple subscribers simultaneously.



Without Fan-Out (Tightly Coupled — Bad)

```
# Every service must be called one by one
# If analytics crashes, fraud detection never runs
call_warehouse()
call_analytics()      # if this crashes...
call_fraud_detection() # ...this never runs
send_email()          # ...and this never runs
```

Problems:

- One failure breaks the entire chain
- Adding a new consumer requires changing this code
- All calls are sequential — slow

With SNS Fan-Out (Loosely Coupled — Good)

```
# Publish once, SNS handles delivery to all subscribers
sns.publish(TopicArn=topic_arn, Message=order_event)
```

```
# Done. All subscribers receive it simultaneously.
```

Benefits:

- One failure does NOT affect others
- Add new consumers without changing existing code
- All deliveries happen in parallel — fast

Real-World Fan-Out Examples

Zomato order placed:

```
SNS "order.placed" event
├── Restaurant app (start cooking)
├── Delivery partner system (find rider)
├── Analytics pipeline (record for reports)
└── SES Lambda (send customer confirmation email)
```

Bank transaction:

```
SNS "transaction.completed" event
├── Account balance update queue
├── Fraud detection queue
├── Audit log queue
└── SMS/Email notification
```

7. Why This Architecture? Real-World Impact

Problem with Traditional Architecture

A traditional monolithic application handles everything in one place:

```
User Request → [Validate + Pay + Notify + Log + Analyze] → Response
```

Issues:

- If the notification service is slow, the user waits
- If any part fails, the whole request fails
- Cannot scale individual parts independently
- Deploying one change requires redeploying everything
- One bug can crash the entire system

How This Serverless Architecture Solves It

Problem	Solution in This Architecture
"Notification is slow, user waits"	Notification runs asynchronously after payment — user gets response immediately
"One failure crashes everything"	Each Lambda is isolated; failures are caught and routed to DLQ

Problem	Solution in This Architecture
"Can't scale parts independently"	Each Lambda scales independently; payment processing can scale to 10x without touching notification code
"Deploying changes is risky"	Update only the Lambda that changed; others are unaffected
"Server costs money when idle"	Lambda costs \$0 when not running

Cost Comparison (Approximate)

Scenario	Traditional Server	This Serverless Architecture
0 orders/month	\$50-200/month (server always on)	~\$0/month
1,000 orders/month	\$50-200/month	< \$1/month
100,000 orders/month	\$200-500/month (need bigger server)	~\$5-10/month
10,000,000 orders/month	\$2,000+/month (multiple servers)	~\$100-200/month

Who Uses This Pattern?

Company	What They Use It For
Amazon	Order processing, fulfillment pipelines
Netflix	Video encoding, recommendation engine
Uber	Trip events, billing, driver dispatch
Swiggy / Zomato	Order management, delivery tracking
Razorpay / Stripe	Payment processing, webhook delivery
Airbnb	Booking workflows, host notifications

8. What Happens When Things Go Wrong?

Retry Logic (Step Functions)

When **ProcessPayment** fails, Step Functions doesn't give up immediately:

```
Attempt 1 → fails → wait 2 seconds
Attempt 2 → fails → wait 4 seconds (2 × 2.0 backoff)
Attempt 3 → fails → wait 8 seconds (4 × 2.0 backoff)
All 3 failed → go to failure notification path
```

This handles **transient failures** — temporary network issues, brief service outages — without losing orders.

Dead Letter Queue (DLQ) — The Safety Net

When payment fails after all retries, the raw error is sent to the **SQS DLQ**:

```
{
  "Error": "Exception",
  "Cause": "PAYMENT_FAILED: Payment gateway declined the transaction"
}
```

Why this matters:

- Engineers can inspect the DLQ to understand what went wrong
- Messages stay in the DLQ for 14 days
- Can be replayed once the underlying issue is fixed
- Nothing is ever silently lost

Without a DLQ (Bad)

Payment fails → error logged to console → nobody notices → customer never gets refund

With a DLQ (Good — Production Standard)

Payment fails → error stored in DLQ → alert fires → engineer investigates → customer gets refund

Every serious production system — banks, payment gateways, e-commerce platforms — has a DLQ strategy.

9. Layman's Summary

Imagine you order food on Swiggy.

The moment you click "Place Order," Swiggy doesn't stop and manually call the restaurant, then call a delivery partner, then send you an email — one at a time. That would be slow and if one call fails, your order is lost.

Instead, Swiggy publishes ONE event — "order placed" — into a system. That system instantly broadcasts it to the restaurant app, the delivery system, the analytics database, and the email service — **all at the same time, independently**.

If the analytics system crashes, your food still gets cooked. If the email fails, your delivery still happens. Each piece works independently.

That is exactly what this architecture does. **Step Functions** manages what happens in what order. **Lambda** does the actual work. **SNS** broadcasts events. **SQS** buffers messages so nothing is lost. **SES** sends reliable emails. **DLQ** catches anything that fails so engineers can fix it.

The result: a system that **scales automatically, costs almost nothing when idle, never loses data, and fails gracefully** — the same way the world's largest tech companies

build their systems.

Document created for Serverless Factory demo — June 2026